



INSTITUTO PROFESIONAL
SAN SEBASTIAN

CREACION DEL PROYECTO DESARROLLO Y PANTALLASOS

1. Creación del proyecto

Se abre terminal en esta ruta:

C:\Users\rosita

```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUE
PS C:\Users\rosita> 
```

Se ejecuta mkdir reservas_salas2_mvc (creación de carpeta)

```
PS C:\Users\rosita> mkdir reservas_salas2_mvc

Directorio: C:\Users\rosita

Mode                LastWriteTime         Length Name
----                -
d-----          29-04-2026      11:20             reservas_salas2_mvc

PS C:\Users\rosita> 
```

cd reservas_salas2_mvc

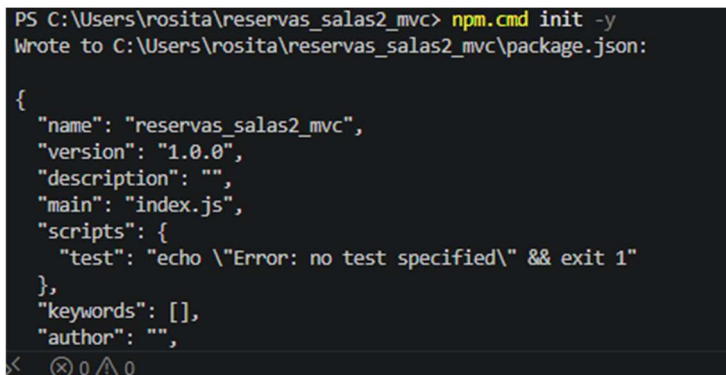
Entrada a la carpeta del proyecto

```
PS C:\Users\rosita> cd reservas_salas2_mvc
PS C:\Users\rosita\reservas_salas2_mvc> 
```

npm init -y Comando para inicializar el proyecto

(Como cause error con este código se uso lo siguiente)

npm.cmd init -y (evita el bloqueo de power Shell)

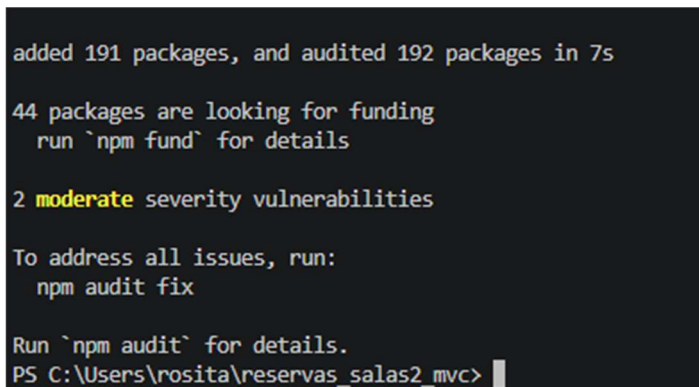
A screenshot of a Windows command prompt window. The title bar is not visible. The prompt shows the command 'npm.cmd init -y' being executed in the directory 'C:\Users\rosita\reservas_salas2_mvc'. The output shows the creation of a 'package.json' file with the following content: { "name": "reservas_salas2_mvc", "version": "1.0.0", "description": "", "main": "index.js", "scripts": { "test": "echo \"Error: no test specified\" && exit 1" }, "keywords": [], "author": "" }. The prompt is now 'PS C:\Users\rosita\reservas_salas2_mvc>'.

INSTALACIÓN DE DEPENDENCIAS

npm install express mysql2 sequelize sequelize-cli dotenv cors jsonwebtoken bcryptjs

(Vuelve a producir error se debe seguir utilizando cmd)

npm.cmd install express mysql2 sequelize sequelize-cli dotenv cors jsonwebtoken bcryptjs
(se utiliza cmd)

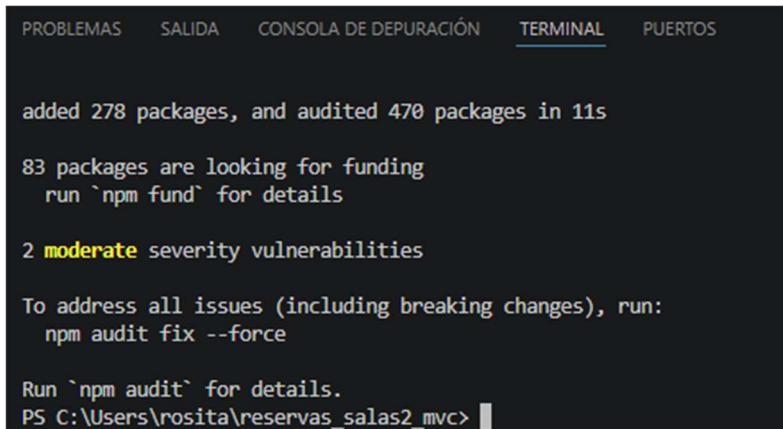
A screenshot of a Windows command prompt window showing the output of 'npm install'. The output includes: 'added 191 packages, and audited 192 packages in 7s', '44 packages are looking for funding', 'run `npm fund` for details', '2 moderate severity vulnerabilities', 'To address all issues, run: npm audit fix', and 'Run `npm audit` for details.'. The prompt is now 'PS C:\Users\rosita\reservas_salas2_mvc>'.

La imagen es solo una parte

Dependencias de desarrollo:

npm install --save-dev nodemon jest supertest
(acá también dio error)

npm.cmd install --save-dev nodemon jest supertest
(se utiliza esto)



```
PROBLEMAS  SALIDA  CONSOLA DE DEPURACIÓN  TERMINAL  PUERTOS

added 278 packages, and audited 470 packages in 11s

83 packages are looking for funding
  run `npm fund` for details

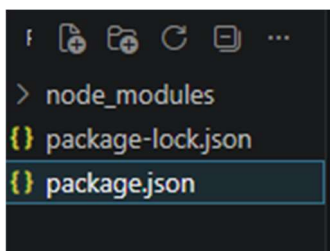
2 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
  npm audit fix --force

Run `npm audit` for details.
PS C:\Users\rosita\reservas_salas2_mvc>
```

Parte de la imagen

2. Abrir Proyecto en Visual.



a. Crear carpetas base:

mkdir src

```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir src

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc

Mode                LastWriteTime         Length
-----
d-----          29-04-2026         11:49

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Mkdir src\models

```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir src\models

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc\src

Mode                LastWriteTime         Length
-----
d-----          29-04-2026         11:51

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Mkdir src\controllers

```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir src\controllers

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc\src

Mode                LastWriteTime         Length
-----
d-----          29-04-2026         11:53

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Mkdir src\routes

```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir src\routes

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc\src

Mode                LastWriteTime         Length
-----
d-----          29-04-2026         11:53

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Mkdir src\middlewares

```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir src\middlewares

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc\src

Mode                LastWriteTime         Length
-----
d-----          29-04-2026         11:54

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Mkdir config

```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir config

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc

Mode                LastWriteTime         Length
-----
d-----          29-04-2026         12:00

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Mkdir migrations

```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir migrations

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc

Mode                LastWriteTime         Length
-----
d-----          29-04-2026         12:01

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Mkdir seeders

```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir seeders

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc

Mode                LastWriteTime         Length
-----
d-----          29-04-2026         11:57

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Mkdir tests

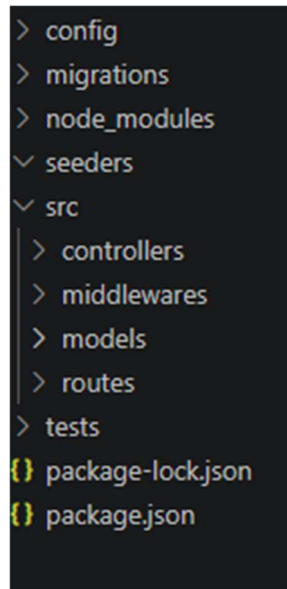
```
PS C:\Users\rosita\reservas_salas2_mv
c> mkdir tests

Directorio: C:\Users\rosita\rese
rvas_salas2_mvc

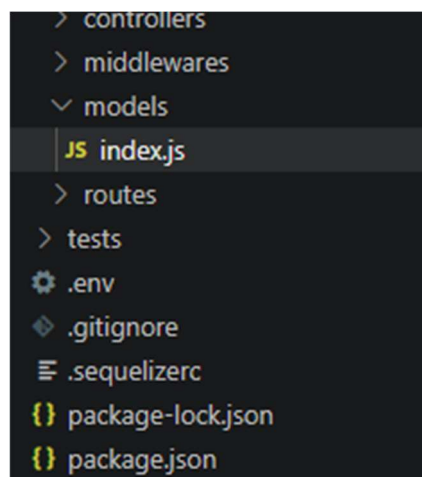
Mode                LastWriteTime         Length
-----
d-----          29-04-2026         12:01

PS C:\Users\rosita\reservas_salas2_mv
c>
```

Resultado final



- Se crea .env
- Se crea .gitignore
- Se crea dentro de config -> config.js
- Se crea src/models/index.js



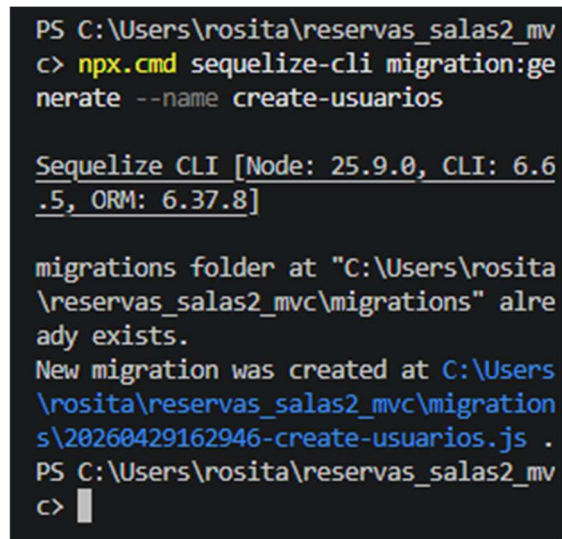
- Se crea src/models/sala.js
- Se crea src/models/reserva.js

b. MIGRACIONES:

1. Migración usuarios:

En la terminal se ejecuta este comando:

`npx.cmd sequelize-cli migration:generate --name create-usuarios`



```
PS C:\Users\rosita\reservas_salas2_mv
c> npx.cmd sequelize-cli migration:generate --name create-usuarios

Sequelize CLI [Node: 25.9.0, CLI: 6.6.5, ORM: 6.37.8]

migrations folder at "C:\Users\rosita\reservas_salas2_mv\migrations" already exists.
New migration was created at C:\Users\rosita\reservas_salas2_mv\migrations\20260429162946-create-usuarios.js .
PS C:\Users\rosita\reservas_salas2_mv
c> |
```

Se crea un archivo dentro de migration 20260429120000-create-usuarios.js

2. Migración Salas

En la terminal se ejecuta:

`npx.cmd sequelize-cli migration:generate --name create-salas`


```

PS C:\Users\rosita\reservas_salas2_mv
c> npx.cmd sequelize-cli migration:generate --name create-salas

Sequelize CLI [Node: 25.9.0, CLI: 6.6.5, ORM: 6.37.8]

migrations folder at "C:\Users\rosita\reservas_salas2_mv\migrations" already exists.
New migration was created at C:\Users\rosita\reservas_salas2_mv\migrations\20260429163336-create-salas.js .
PS C:\Users\rosita\reservas_salas2_mv
c>

```

Se crea un archivo dentro
2026042916xxxx-create-salas.js

3. Migración reserva

En la terminal ejecuta:

npx.cmd sequelize-cli migration:generate --name create-reservas

```

PS C:\Users\rosita\reservas_salas2_mv
c> npx.cmd sequelize-cli migration:generate --name create-reservas

Sequelize CLI [Node: 25.9.0, CLI: 6.6.5, ORM: 6.37.8]

migrations folder at "C:\Users\rosita\reservas_salas2_mv\migrations" already exists.
New migration was created at C:\Users\rosita\reservas_salas2_mv\migrations\20260429163636-create-reservas.js .
PS C:\Users\rosita\reservas_salas2_mv
c>

```

Se crea un archivo dentro 2026042916xxxx-create-reservas.js

c. PROBANDO LAS TABLAS

En la terminal se ejecuta:

```
npx.cmd sequelize-cli db:migrate
```

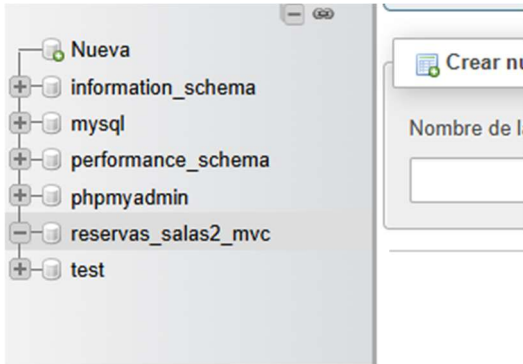
```
== create-usuarios: migrated
```

```
== create-salas: migrated
```

```
== create-reservas: migrated
```

Al abrir Mysql se ven las tablas :

usuarios
salas
reservas
SequelizeMeta



MIGRACION EJECUTADA

```
PS C:\Users\rosita\reservas_salas2_mv
c> npx.cmd sequelize-cli db:migrate

Sequelize CLI [Node: 25.9.0, CLI: 6.6
.5, ORM: 6.37.8]

◇ injected env (8) from .env // tip:
{ multiple files { path: ['.env.local
', '.env'] } }
Loaded configuration file "config\con
fig.js".
Using environment "development".
== 20260429162946-create-usuarios: mi
grating =====
== 20260429162946-create-usuarios: mi
grated (0.035s)

== 20260429163336-create-salas: migra
ting =====
== 20260429163336-create-salas: migra
ted (0.019s)

== 20260429163636-create-reservas: mi
grating =====
== 20260429163636-create-reservas: mi
grated (0.034s)

PS C:\Users\rosita\reservas_salas2_mv
c>
```

TABLAS MYSQL

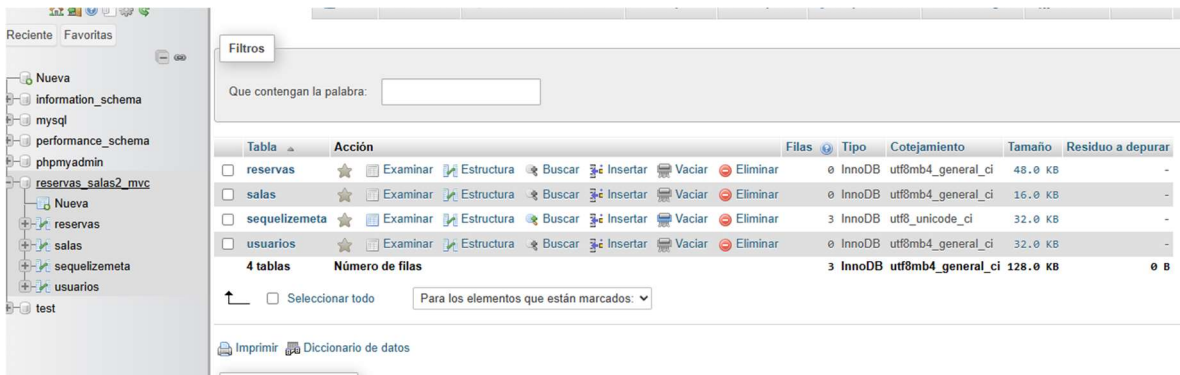


Tabla	Acción	Filas	Tipo	Cotejamiento	Tamaño	Residuo a depurar
☐ reservas	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	48.0 KB	-
☐ salas	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	16.0 KB	-
☐ sequelizemeta	Examinar Estructura Buscar Insertar Vaciar Eliminar	3	InnoDB	utf8_unicode_ci	32.0 KB	-
☐ usuarios	Examinar Estructura Buscar Insertar Vaciar Eliminar	0	InnoDB	utf8mb4_general_ci	32.0 KB	-
4 tablas		Número de filas		3 InnoDB	utf8mb4_general_ci	128.0 KB

↑ ☐ Seleccionar todo Para los elementos que están marcados: ▼

Imprimir Diccionario de datos

d. Crear seeder

En la terminal ejecutar:

`npx.cmd sequelize-cli seed:generate --name demo-usuarios`

```
PS C:\Users\rosita\reservas_salas2_mv
c> npx.cmd sequelize-cli seed:generat
e --name demo-usuarios

Sequelize CLI [Node: 25.9.0, CLI: 6.6
.5, ORM: 6.37.8]

seeders folder at "C:\Users\rosita\re
servas_salas2_mvc\seeders" already ex
ists.
New seed was created at C:\Users\rosi
ta\reservas_salas2_mvc\seeders\202604
29165448-demo-usuarios.js .
PS C:\Users\rosita\reservas_salas2_mv
c> |
```

Crea un archivo dentro 20260429xxxxxx-demo-usuarios.js

SEEDER SALAS

En la terminal se ejecuta:

`npx.cmd sequelize-cli seed:generate --name demo-salas`

```

PS C:\Users\rosita\reservas_salas2_mv
c> npx.cmd sequelize-cli seed:generate
--name demo-salas

Sequelize CLI [Node: 25.9.0, CLI: 6.6
.5, ORM: 6.37.8]

seeders folder at "C:\Users\rosita\re
servas_salas2_mvc\seeders" already ex
ists.
New seed was created at C:\Users\rosi
ta\reservas_salas2_mvc\seeders\202604
29170332-demo-salas.js .
PS C:\Users\rosita\reservas_salas2_mv
c>

```

Crea otro archivo dentro 20260429xxxxxx-demo-salas.js

SEEDERS RESERVAS

En la terminal se ejecuta:

npx.cmd sequelize-cli seed:generate --name demo-reservas

```

PS C:\Users\rosita\reservas_salas2_mv
c> npx.cmd sequelize-cli seed:generate
--name demo-reservas

Sequelize CLI [Node: 25.9.0, CLI: 6.6
.5, ORM: 6.37.8]

seeders folder at "C:\Users\rosita\re
servas_salas2_mvc\seeders" already ex
ists.
New seed was created at C:\Users\rosi
ta\reservas_salas2_mvc\seeders\202604
29170651-demo-reservas.js .
PS C:\Users\rosita\reservas_salas2_mv
c>

```

Se crea un archivo dentro 20260429xxxxxx-demo-reservas.js

PROBAR O VALIDAR LOS SEEDERS

Ahora en la terminal se ejecuta:

`npx.cmd sequelize-cli db:seed:all`

```
PS C:\Users\rosita\reservas_salas2_mv
c> npx.cmd sequelize-cli db:seed:all

Sequelize CLI [Node: 25.9.0, CLI: 6.6
.5, ORM: 6.37.8]

◇ injected env (8) from .env // tip:
  ~ auth for agents [www.vestauth.com]
Loaded configuration file "config\con
fig.js".
Using environment "development".
== 20260429165448-demo-usuarios: migr
ating =====
== 20260429165448-demo-usuarios: migr
ated (0.153s)

== 20260429170332-demo-salas: migrati
ng =====
== 20260429170332-demo-salas: migrate
d (0.007s)

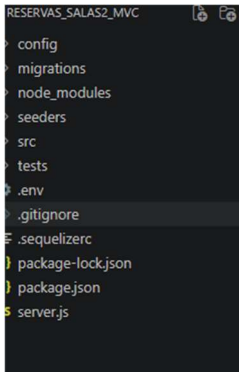
== 20260429170651-demo-reservas: migr
ating =====
== 20260429170651-demo-reservas: migr
ated (0.008s)

PS C:\Users\rosita\reservas_salas2_mv
c> 
```

- Se crea src/app.js

```
▼ src
  > controllers
  > middlewares
  > models
  > routes
  JS app.js
▼ tests
  .env
  .gitignore
  JS .sequelizerc
  {} package-lock.json
  {} package.json
```

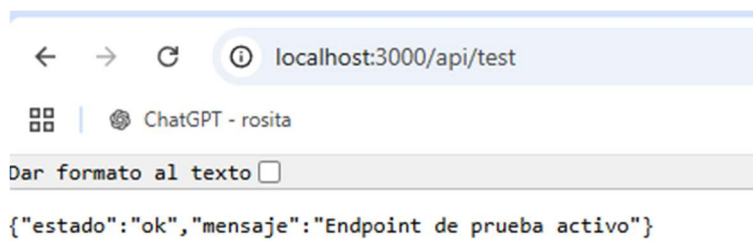
- Se crea server.js



e. PRUEBA SERVIDOR

Se ejecuta este comando para levantar el servidor

npm.cmd run dev



<http://localhost:3000>

<http://localhost:3000/api/test>

SE CORRIGE ERROR EN LA TERMINAL

Durante el desarrollo apareció un error al intentar levantar el servidor. El mensaje que mostró la terminal fue:

```
TypeError: require(...) is not a function
```

El error apuntaba al archivo `src/models/index.js`, específicamente a la parte donde Sequelize carga los modelos del proyecto. Esto indicaba que uno de los archivos dentro de la carpeta `models` no estaba siendo leído correctamente como función.

Para revisar el problema, primero se verificaron los archivos existentes dentro de la carpeta `src/models` usando el siguiente comando:

```
cmd /c dir src\models /b
```

El resultado mostró que estaban los archivos `index.js`, `reserva.js`, `sala.js` y `usuario.js`. Por lo tanto, el problema no era que faltara un archivo o que estuviera mal nombrado.

Luego se realizó una prueba desde la terminal para revisar qué tipo de dato estaba exportando cada modelo:

```
node -e "[ 'usuario', 'sala', 'reserva' ].forEach(m=>console.log(m, typeof require('./src/models/'+m)))"
```

El resultado indicó que `usuario.js` y `sala.js` estaban exportando correctamente una función, pero `reserva.js` estaba exportando un objeto. Esto permitió identificar que el problema estaba en el archivo `reserva.js`.

El error ocurría porque los modelos de Sequelize debían exportarse como una función, para que `index.js` pudiera cargarlos correctamente. Se corrigió el contenido del archivo `reserva.js`, dejándolo con la estructura adecuada del modelo.

Después de corregirlo, se volvió a ejecutar la prueba en la terminal y esta vez los tres modelos aparecieron como `function`:

```
usuario function  
sala function  
reserva function
```

Con esto se confirmó que los modelos estaban correctamente exportados. Luego se levantó nuevamente el servidor y la API respondió correctamente en `localhost`, tanto en la ruta principal como en el endpoint de prueba.

- Se crea el archivo

src/controllers/authController.js

4. Creación de la ruta del login

Dentro de src/routes se crea el archivo authRoutes.js, para luego conectar la ruta en app.js

```
const authRoutes = require('./routes/authRoutes');
```

a. Prueba de login en Postman

Se ejecuta:

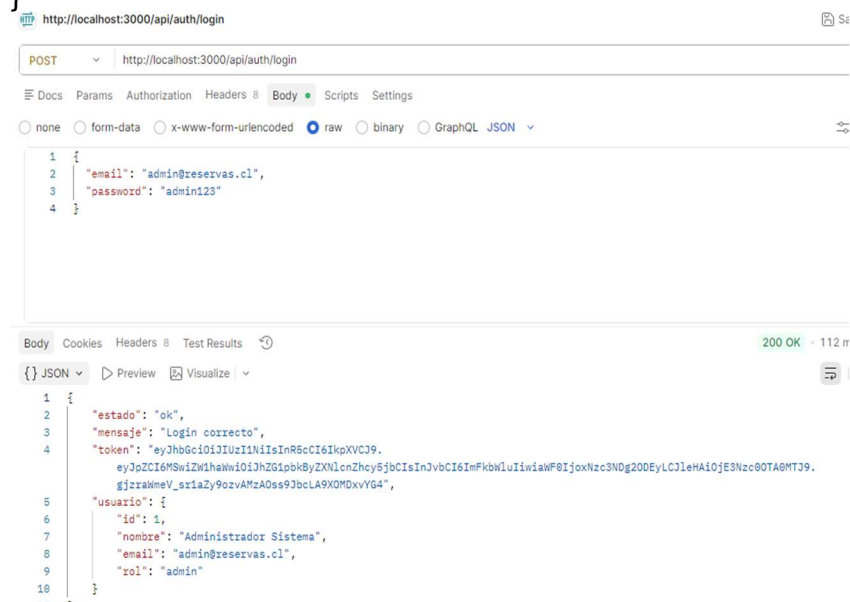
Método: POST

URL: http://localhost:3000/api/auth/login

Body → raw → JSON

Ejemplo de prueba :

```
{
  "email": "admin@reservas.cl",
  "password": "admin123"
}
```



Valor :

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJhZG1pbkByZXNlcnZhcj5jbCI6ImFkbWluliwiaWF0IjoxNzc3NjM4MzU2LCJleHAiOiE3Nzc2NDE5NTZ9.i6V95ozU8JVkd_M3EW8XVIIIMhGfxhLwQjGLLuLnfvEI

b. Prueba login incorrecto:

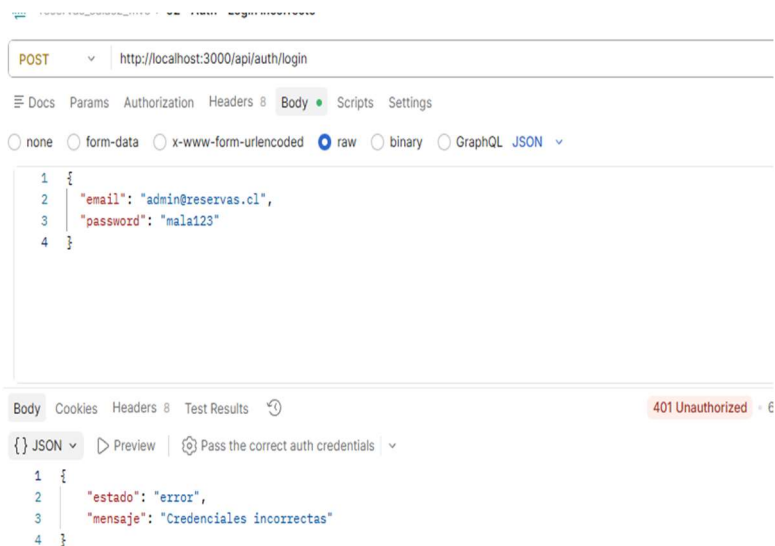
Método: POST

URL: <http://localhost:3000/api/auth/login>

Body → raw → JSON:

Ejemplo de prueba:

```
{
  "email": "admin@reservas.cl",
  "password": "mala123"
}
```



- Se crea el archivo `src/middlewares/authMiddleware.js`

c. Probar Token en postman

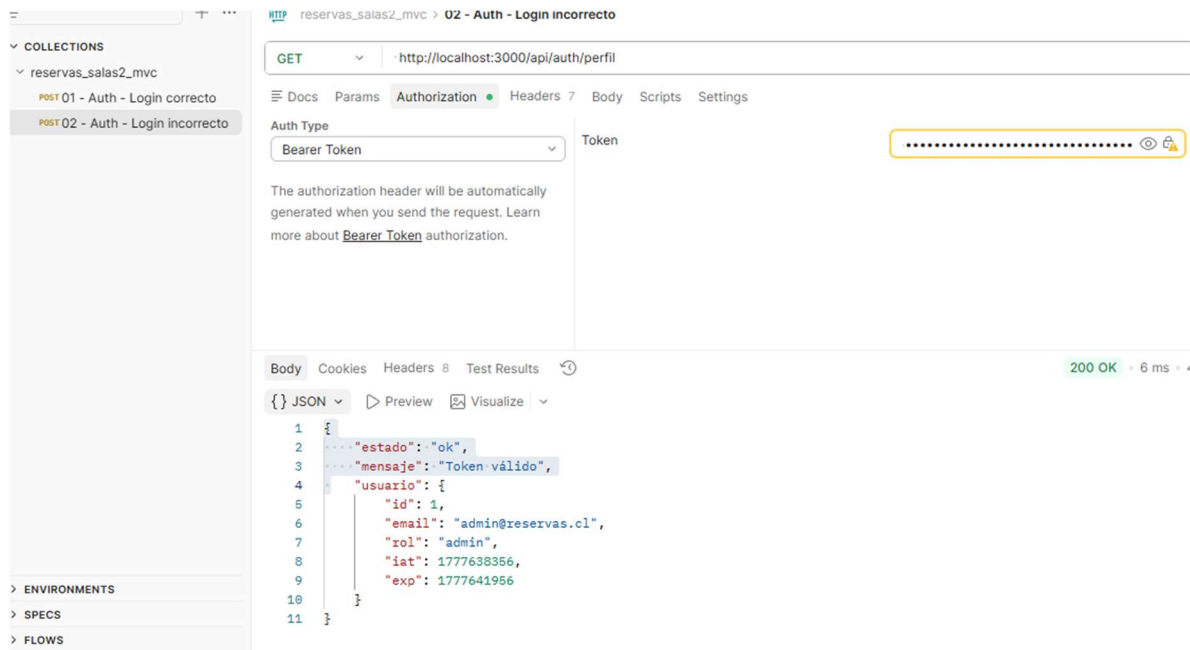
Método: GET

URL: <http://localhost:3000/api/auth/perfil>

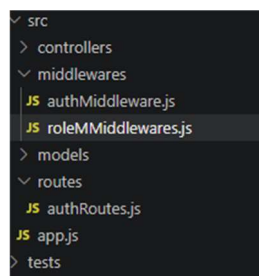
Luego en Postman ir Authorization → Type → Bearer Token

Y pegar el valor:

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6Im5wZW1haWwiOiJhZG1pbkByZXNlcnZhcj5jbCI6ImFkbWluliwiaWF0IjoxNzc3NjM4MzU2LCJleHAiOiE3Nzc2NDE5NTZ9.i6V95ozU8JVkd_M3EW8XVIIMhGfxhLwQjGLLuLnfvEI



- Se crea este archivo `src/middlewares/roleMiddleware.js`



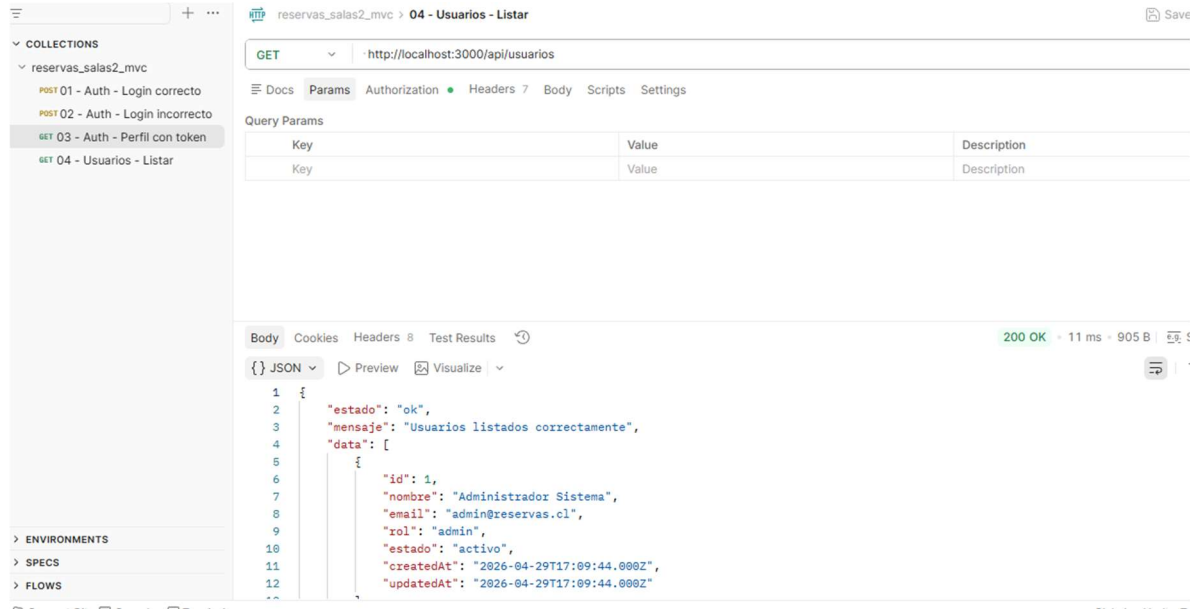
- Se crea el archivo `src/routes/usuarioRoutes.js`

5. Prueba crud usuarios

- Listar usuarios (se crea una petición)

Método : URL : GET <http://localhost:3000/api/usuarios>

Luego en Postman Authorization → Auth Type: Bearer Token
Se utiliza el token del admin que se obtuvo en el login



- Prueba usuario

Método: POST

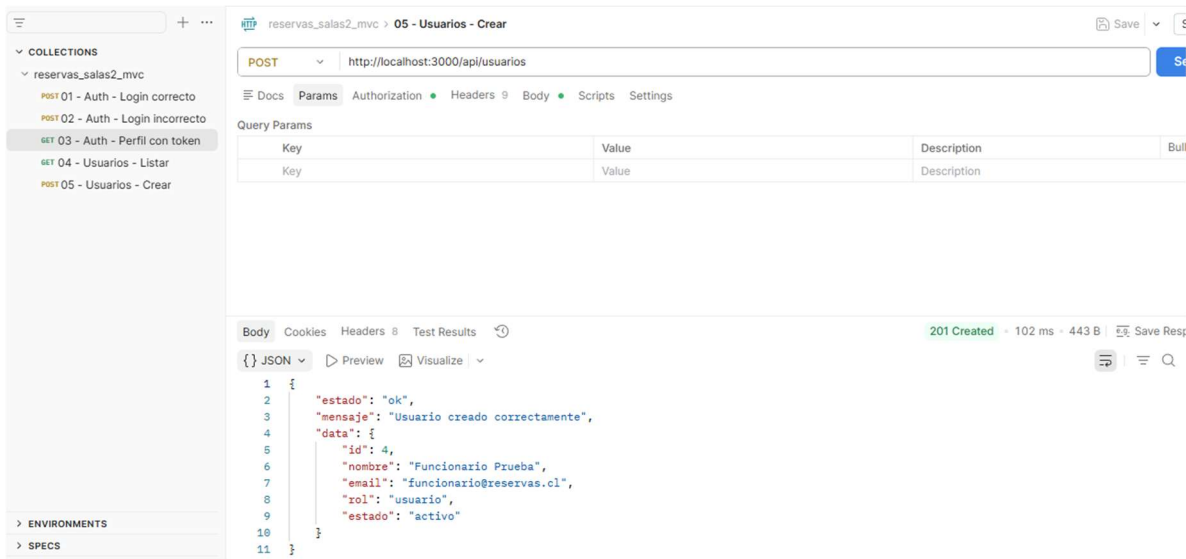
URL: `http://localhost:3000/api/usuarios`

Authorization: Bearer Token

En Body → raw → JSON

Ejemplo prueba:

```
{
  "nombre": "Funcionario Prueba",
  "email": "funcionario@reservas.cl",
  "password": "funcionario123",
  "rol": "usuario"
}
```



- **Prueba usuario duplicado**

Método: POST

URL: http://localhost:3000/api/usuarios

Authorization: Bearer Token del admin

Body → raw → JSON:

Ejemplo

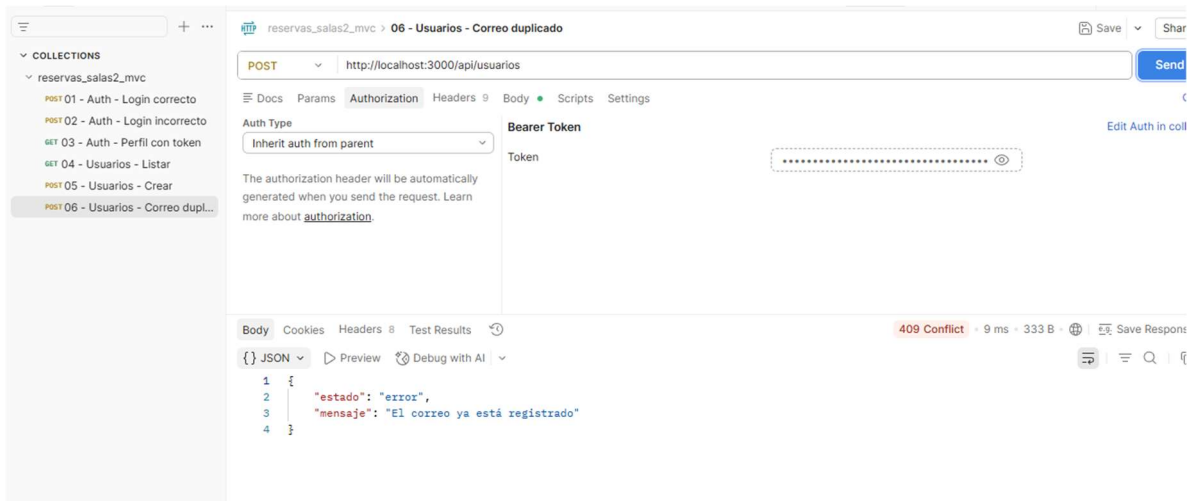
```

{
  "nombre": "Funcionario Prueba",
  "email": "funcionario@reservas.cl",
  "password": "funcionario123",
  "rol": "usuario"
}

```

Valor nuevo

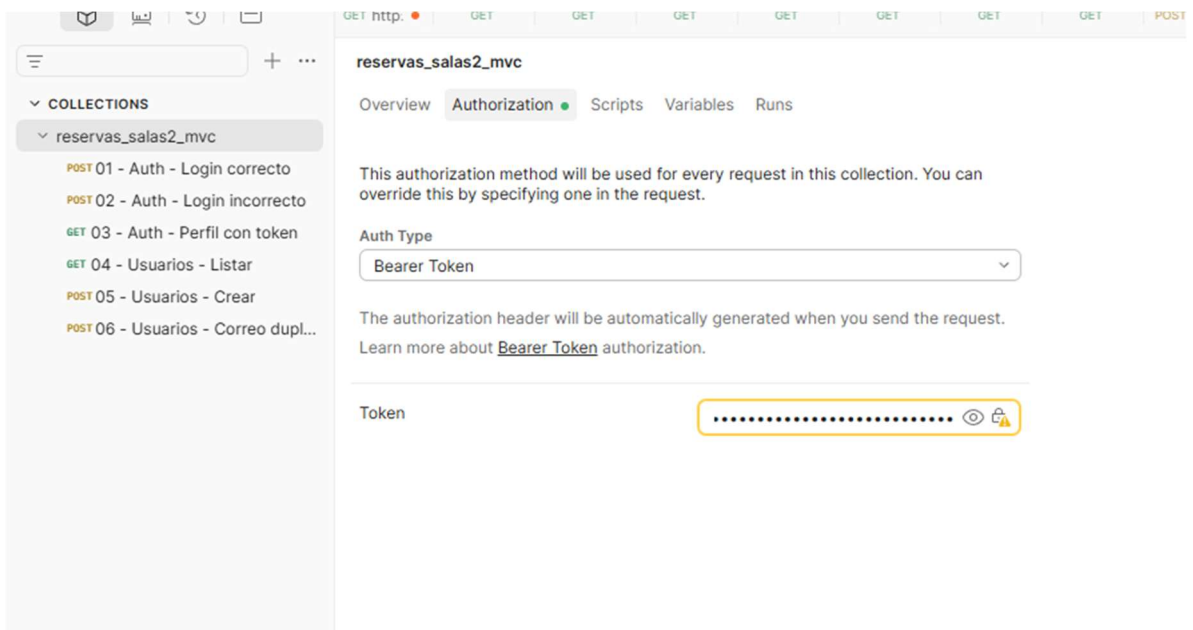
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJhZG1pbkByZXNlcnZhcy5jbCI6ImFkbWluliwiaWF0IjoxNzc3NjQyOTUzLCJleHAiOiE3Nzc2NDY1NTN9.ZXRnUcvKuoAYsqA2je4EgURJX3eD7hnWr7CDRSEHXNk



ERROR

Al probar el usuario duplicado apareció primero un error 401 porque el token estaba expirado. Se solucionó generando un nuevo token desde el login y configurándolo en la colección de Postman. Luego la request tomó ese token con “Inherit auth from parent” y la prueba respondió correctamente con 409 Conflict, indicando que el correo ya estaba registrado.

Debido que los tokens JWT tienen un tiempo de duración limitado y no se actualizan automáticamente en todas las pruebas de Postman



- **Prueba PUT usuario**

Método: PUT

URL: <http://localhost:3000/api/usuarios/3>

Authorization: Inherit auth from parent

Usar Body → raw → JSON, pega:

```
{
  "nombre": "Funcionario Prueba Actualizado",
  "email": "funcionario.actualizado@reservas.cl",
  "password": "funcionario456",
  "rol": "usuario"
}
```

The screenshot displays a REST client interface with a sidebar on the left and a main workspace on the right. The sidebar, titled 'COLLECTIONS', lists several API endpoints under the 'reservas_salas2_mvc' collection, with 'PUT 07 - Usuarios - Actualizar' selected. The main workspace shows a PUT request to 'http://localhost:3000/api/usuarios/3'. The 'Body' tab is active, showing the request body in raw JSON format. Below the request, the response is shown in the 'Body' tab, displaying a JSON object with status 'ok', a success message, and user details. The status bar at the bottom right indicates a '200 OK' response.

Request:

```
PUT http://localhost:3000/api/usuarios/3
```

Request Body (raw JSON):

```
{
  "nombre": "Funcionario Prueba Actualizado",
  "email": "funcionario.actualizado@reservas.cl",
  "password": "funcionario456",
  "rol": "usuario"
}
```

Response (JSON):

```
{
  "estado": "ok",
  "mensaje": "Usuario actualizado correctamente",
  "data": {
    "id": 3,
    "nombre": "Funcionario Prueba Actualizado",
    "email": "funcionario.actualizado@reservas.cl",
    "rol": "usuario",
    "estado": "inactivo"
  }
}
```

Status: 200 OK

- **Prueba PATCH estado usuario**

Método: PATCH

URL: <http://localhost:3000/api/usuarios/3/estado>

Authorization: Inherit auth from parent

En Body → raw → JSON

Ejemplo

```
{  
  "estado": "inactivo"  
}
```

The screenshot displays a REST client interface with the following components:

- Left Panel (Collections):** Lists several API endpoints. The selected endpoint is "PATCH 08 - Usuarios - Cambiar estado".
- Top Bar:** Shows the method "PATCH" and the URL "http://localhost:3000/api/usuarios/3/estado".
- Query Params:** A table with two columns: "Key" and "Value".
- Body:** The response is shown in JSON format. The status is "200 OK" with a response time of "9 ms" and a size of "480 B".

The JSON response body is as follows:

```
1 {  
2   "estado": "ok",  
3   "mensaje": "Estado del usuario actualizado correctamente",  
4   "data": {  
5     "id": 3,  
6     "nombre": "Funcionario Prueba Actualizado",  
7     "email": "funcionario.actualizado@reservas.cl",  
8     "rol": "usuario",  
9     "estado": "inactivo"  
10  }  
11 }
```

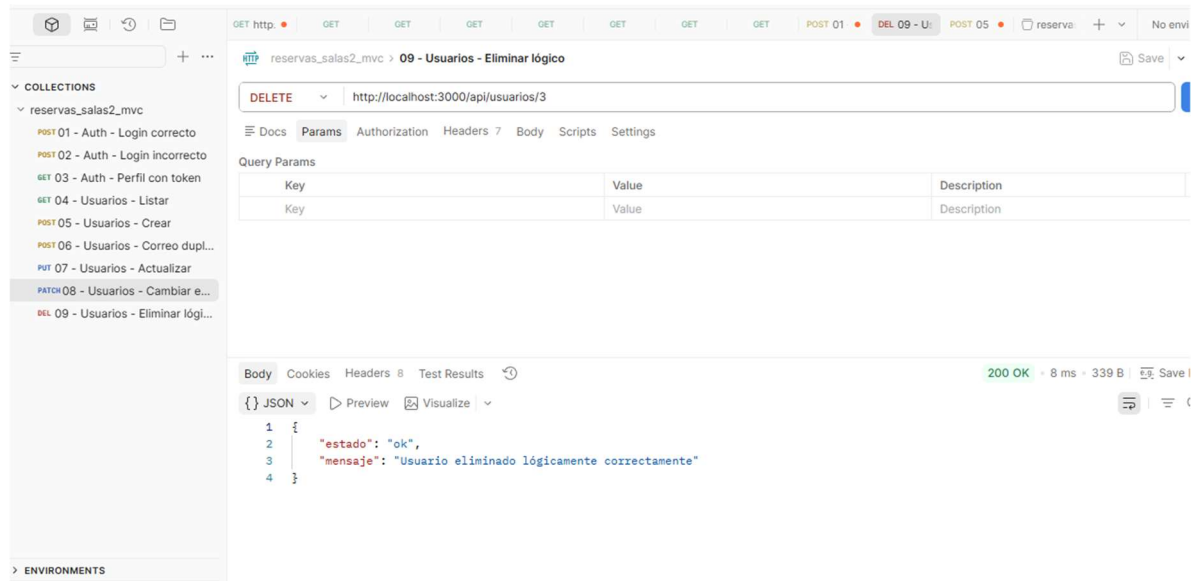
- **Prueba DELETE**

Método: DELETE

URL: <http://localhost:3000/api/usuarios/3>

Authorization: Inherit auth from parent

Body: none



6. Salas

- Crear src/controllers/slaController.js
- Crear src/routes/salaRoutes.js
- Crear src/controllers/salaController.js

Se revisa si la terminal est\u00e1 corriendo.


```

e
Servidor ejecutándose en http://localhost:3000
[nodemon] restarting due to changes...
[nodemon] starting `node server.js`
◇ injected env (8) from .env // tip: ~ a
th for agents [www.vestauth.com]
◇ injected env (0) from .env // tip: % m
ltiple files { path: ['.env.local', '.env
'] }
◇ injected env (0) from .env // tip: % m
ltiple files { path: ['.env.local', '.env
'] }
Conexión a MySQL establecida correctamen
e
Servidor ejecutándose en http://localhost:3000
[]

```

- **Prueba en Postman salas**

Método: GET

URL: http://localhost:3000/api/salas

Authorization: Inherit auth from parent

The screenshot shows the Postman interface with a collection named 'reservas_salas2_mvc'. The selected request is 'GET 10 - Salas - Listar'. The URL is 'http://localhost:3000/api/salas'. The response is a 200 OK status with a response time of 24 ms. The JSON body is as follows:

```

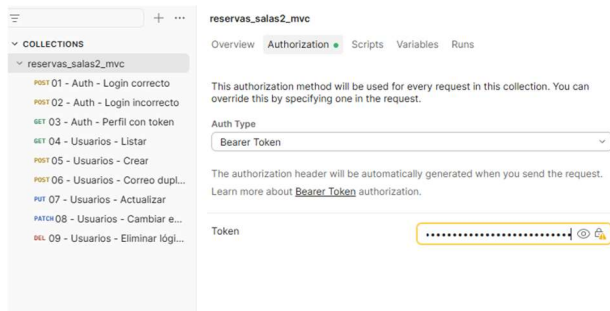
{
  "estado": "ok",
  "mensaje": "Salas listadas correctamente",
  "data": [
    {
      "id": 1,
      "nombre": "Sala Reuniones A",
      "capacidad": 10,
      "ubicacion": "Primer piso",
      "estado": "disponible",
      "createdAt": "2026-04-29T17:09:44.000Z",
      "updatedAt": "2026-04-29T17:09:44.000Z"
    }
  ]
}

```

Valor nuevo

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJhZG1pbkByZXNlcnZhcj5jbCI6ImFkbWluliwiaWF0IjoxNzc3NjQ3MDMwLCJleHAiOiE3Nzc2NTA2MzB9.q8h31-hAV8eO_86p_2GhT00b1nl65vnTSpoSFDrfAnA

Se cambia valor



- **Prueba crear sala**

Método: POST

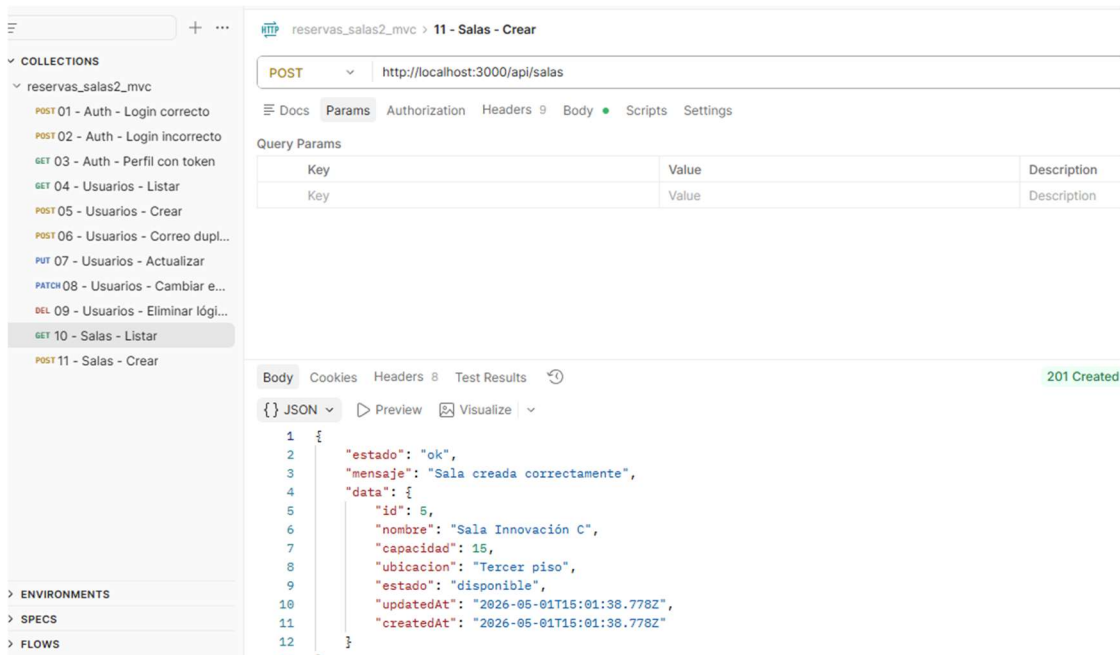
URL: `http://localhost:3000/api/salas`

Authorization: Inherit auth from parent

En Body → raw → JSON:

Ejemplo

```
{
  "nombre": "Sala Innovación C",
  "capacidad": 15,
  "ubicacion": "Tercer piso",
  "estado": "disponible"
}
```



- **Prueba actualizar sala**

Método: PUT

URL: <http://localhost:3000/api/salas/5>

Authorization: Inherit auth from parent

Body → raw → JSON:

```
{
  "nombre": "Sala Innovación C Actualizada",
  "capacidad": 20,
  "ubicacion": "Tercer piso - Ala Norte",
  "estado": "disponible"
}
```

The screenshot shows the Postman interface for a PUT request to `http://localhost:3000/api/salas/5`. The left sidebar lists a collection named `reservas_salas2_mvc` with 12 items. The main panel shows the request details, including the URL, method (PUT), and a table for Query Params. The response tab is active, showing a 200 OK status and a JSON body with the following structure:

```
{
  "estado": "ok",
  "mensaje": "Sala actualizada correctamente",
  "data": {
    "id": 5,
    "nombre": "Sala Innovación C Actualizada",
    "capacidad": 20,
    "ubicacion": "Tercer piso - Ala Norte",
    "estado": "disponible",
    "createdAt": "2026-05-01T15:01:38.000Z",
    "updatedAt": "2026-05-01T15:07:24.000Z"
  }
}
```

- **Prueba cambiar estado de sala**

Método: PATCH

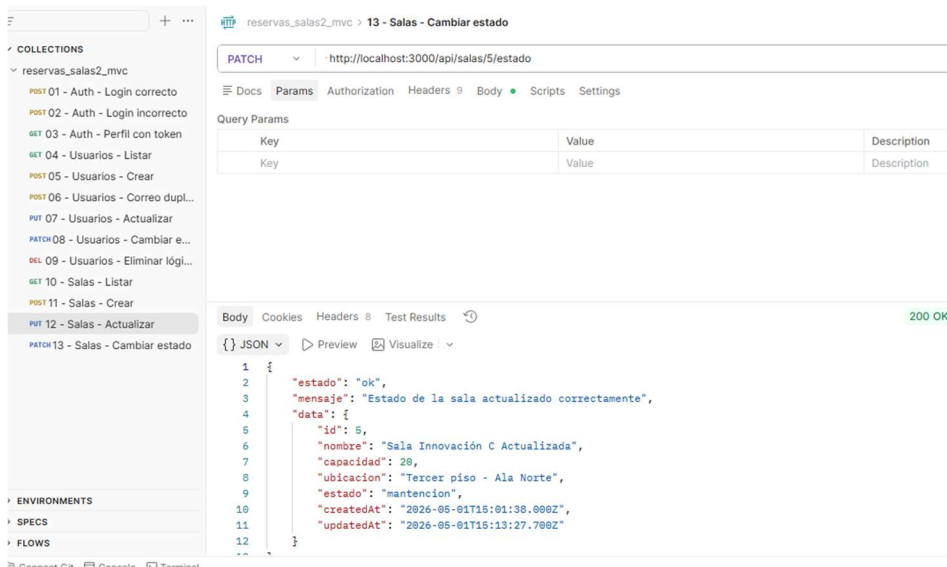
URL: `http://localhost:3000/api/salas/5/estado`

Authorization: Inherit auth from parent

Ejemplo:

Body → raw → JSON:

```
{
  "estado": "mantencion"
}
```



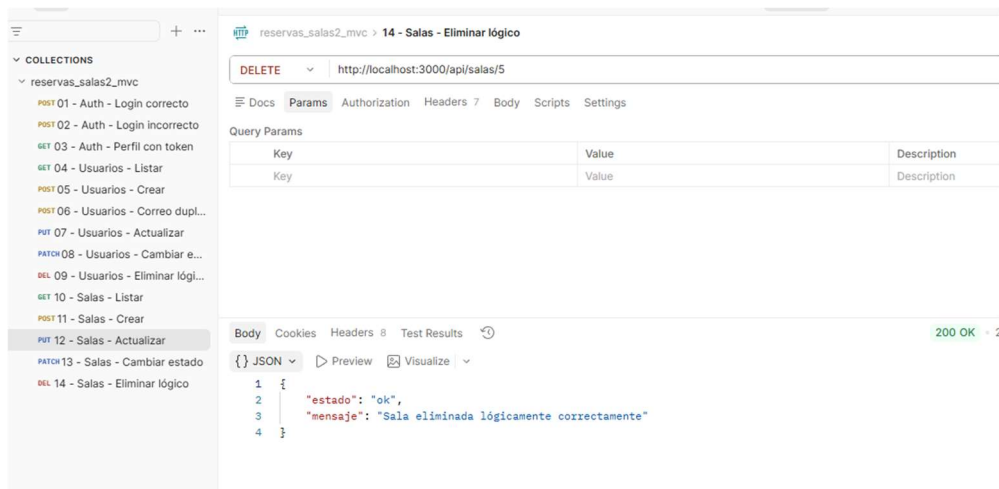
- **Prueba DELETE sala**

Método: DELETE

URL: http://localhost:3000/api/salas/5

Authorization: Inherit auth from parent

Body: none



7. Modulo Reservas

- Se crea src/controllers/reservaController.js
- Se crea src/routes/reservaRoutes.js

Se revisa terminal, para comprobar que nodemon este corriendo

Ahora mira la terminal. Si nodemon está corriendo, debería reiniciar solo.

```
:3000
[nodemon] restarting due to changes...
[nodemon] starting 'node server.js'
◇ injected env (8) from .env // tip: ✦ encrypted .env [www.dotenvx.com]
◇ injected env (0) from .env // tip: ⓘ multiple files { path: ['.env.local', '.env'] }
◇ injected env (0) from .env // tip: ⓘ suppress logs { quiet: true }
Conexión a MySQL establecida correctamente
Servidor ejecutándose en http://localhost:3000
[nodemon] restarting due to changes...
[nodemon] starting 'node server.js'
◇ injected env (8) from .env // tip: ✦ secrets for agents [www.dotenvx.com]
◇ injected env (0) from .env // tip: ⓘ custom filepath { path: '/custom/path/.env' }
◇ injected env (0) from .env // tip: ⓘ enable debugging { debug: true }
Conexión a MySQL establecida correctamente
Servidor ejecutándose en http://localhost:3000
[]
```

TRANSACCIONES

En el archivo `src/controllers/reservaController.js` se agregó el uso de transacciones con Sequelize para reforzar las operaciones principales del módulo de reservas.

Este ajuste se aplicó en las acciones de crear reserva, actualizar reserva, cambiar estado y cancelar reserva. La finalidad fue proteger estas operaciones para que los cambios se guarden correctamente solo cuando el proceso termina sin errores.

Cuando la operación se realiza correctamente, se ejecuta `commit`, lo que confirma los cambios en la base de datos. En cambio, si ocurre un error durante el proceso, se ejecuta `rollback`, lo que permite revertir la operación y evitar que queden datos incompletos o inconsistentes.

Después de este ajuste se volvieron a probar las principales requests del módulo reservas en Postman, verificando que listar, crear, actualizar, cambiar estado, cancelar y validar horarios siguieran funcionando correctamente.

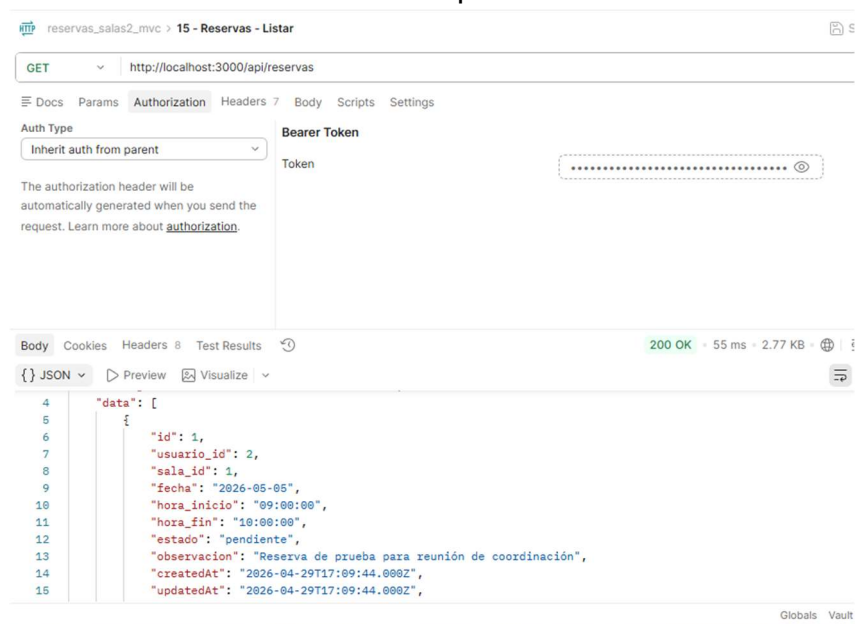
Request	Método	Resultado
15 - Reservas - Listar	GET	200 OK
16 - Reservas - Crear	POST	201 Created
17 - Reservas - Horario ocupado	POST	409 Conflict
18 - Reservas - Fuera de horario	POST	400 Bad Request
19 - Reservas - Actualizar	PUT	200 OK
20 - Reservas - Cambiar estado	PATCH	200 OK
21 - Reservas - Cancelar	DELETE	200 OK

- **Prueba listar reservas**

Método: GET

URL: http://localhost:3000/api/reservas

Authorization: Inherit auth from parent



- **Post reserva**

Método: POST

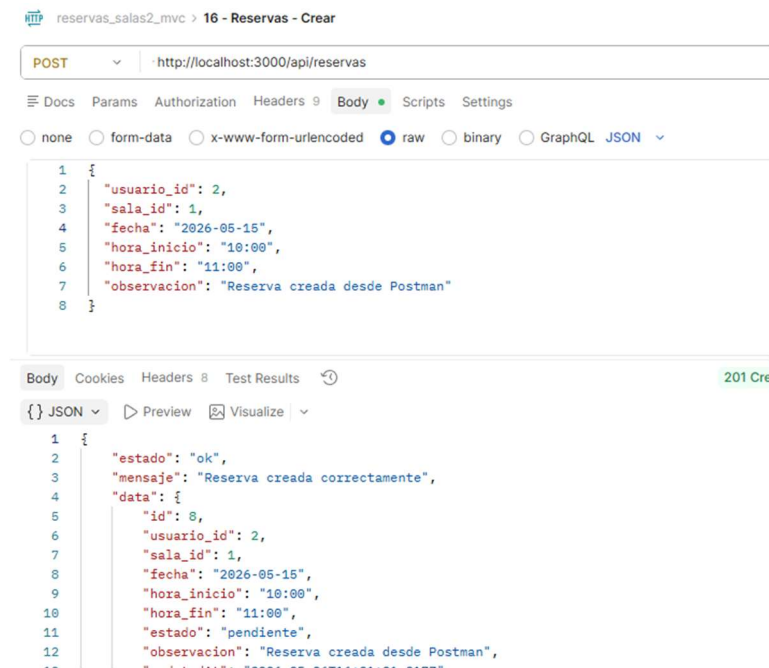
URL: http://localhost:3000/api/reservas

Authorization: Inherit auth from parent

En Body → raw → JSON

Ejemplo

```
{
  "usuario_id": 2,
  "sala_id": 1,
  "fecha": "2026-05-07",
  "hora_inicio": "10:00",
  "hora_fin": "11:00",
  "observacion": "Reserva creada desde Postman"
}
```



- **Prueba reserva duplicada/ sala ocupada**

Método: POST

URL: `http://localhost:3000/api/reservas`

Authorization: Inherit auth from parent

En Body → raw → JSON pega:

Ejemplo:

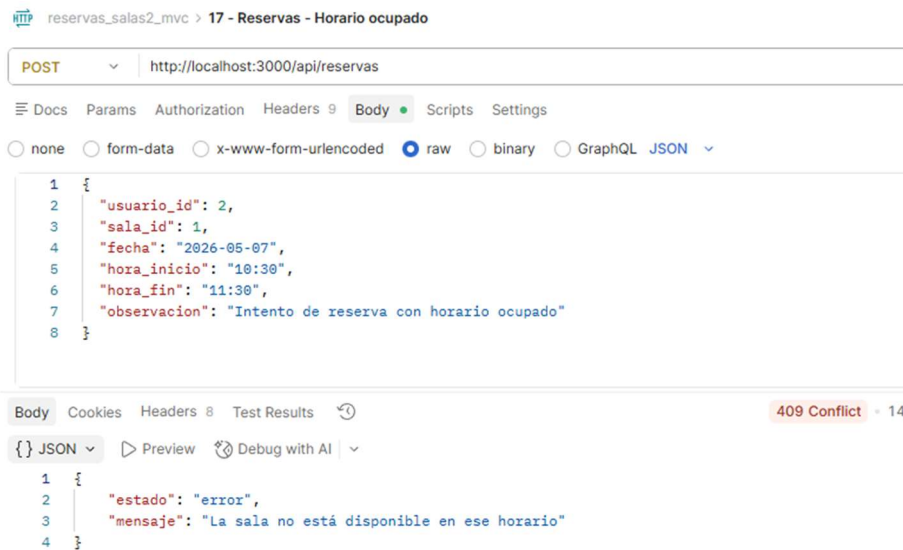
```
{
  "usuario_id": 2,
  "sala_id": 1,
  "fecha": "2026-05-07",
  "hora_inicio": "10:30",
  "hora_fin": "11:30",
  "observacion": "Reserva creada desde Postman"
}
```



```
"observacion": "Intento de reserva con horario ocupado"
}
```

VALOR NUEVO

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwiZW1haWwiOiJhZG1pbkByZXNlcnZhcj5jbCIsInJvbCI6ImFkbWluliwiaWF0IjoxNzc3NjUxMjU2LCJleHAiOjE3Nzc2NTQ4NTZ9.qHO1hZz2noa9VRJU_DyfuCyQpOdEN8cnZdGzrqs_M9o



- **Prueba reserva fuera del horario permitido**

Método: POST

URL: `http://localhost:3000/api/reservas`

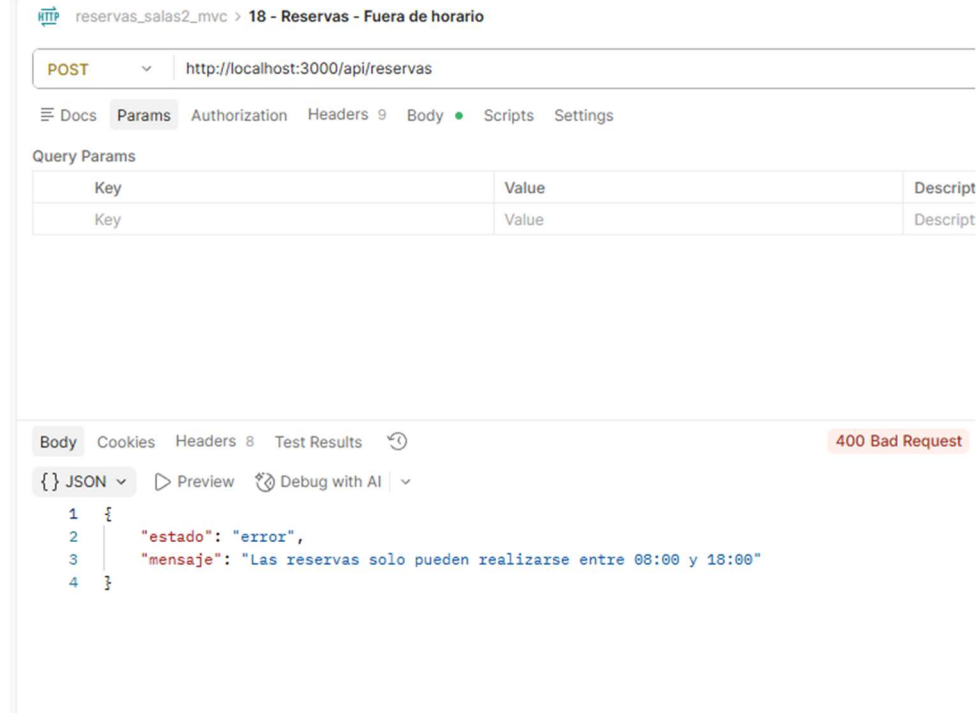
Authorization: igual que la request 17

En **Body** → **raw** → **JSON**

Ejemplo:

```
{
  "usuario_id": 2,
  "sala_id": 1,
  "fecha": "2026-05-08",
  "hora_inicio": "19:00",
  "hora_fin": "20:00",
}
```

"observacion": "Intento fuera de horario permitido"



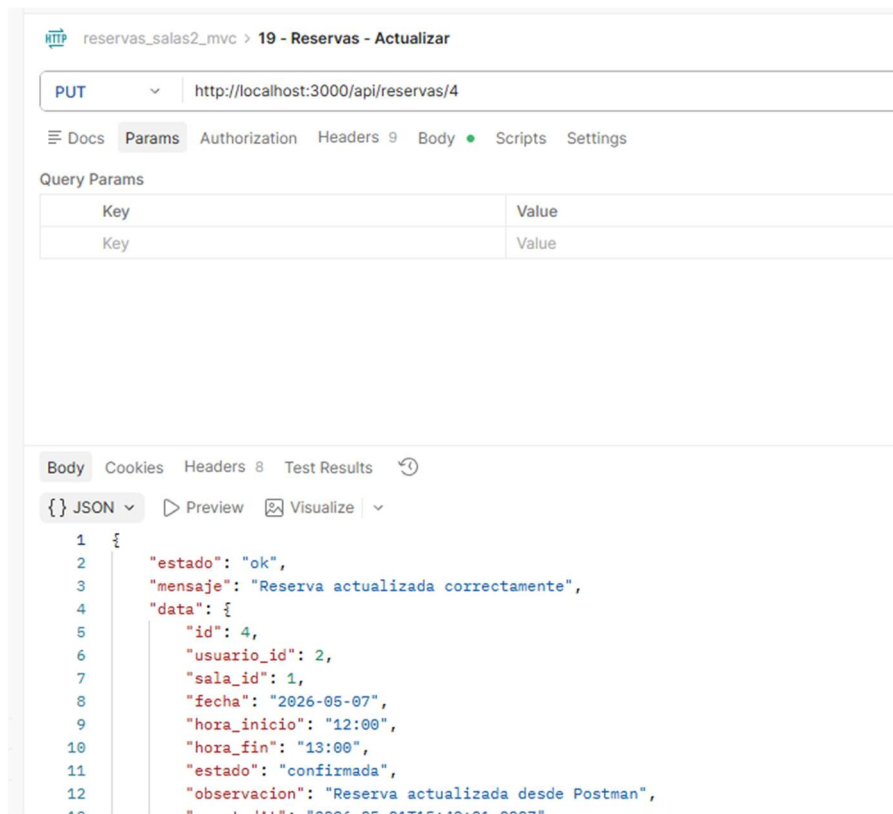
- **Prueba actualizar reserva**

Metodo PUT

URL : `http://localhost:3000/api/reservas/4`

Body → raw → JSON:

```
{
  "usuario_id": 2,
  "sala_id": 1,
  "fecha": "2026-05-07",
  "hora_inicio": "12:00",
  "hora_fin": "13:00",
  "estado": "confirmada",
  "observacion": "Reserva actualizada desde Postman"
}
```



- **Prueba cambiar estado de reserva**

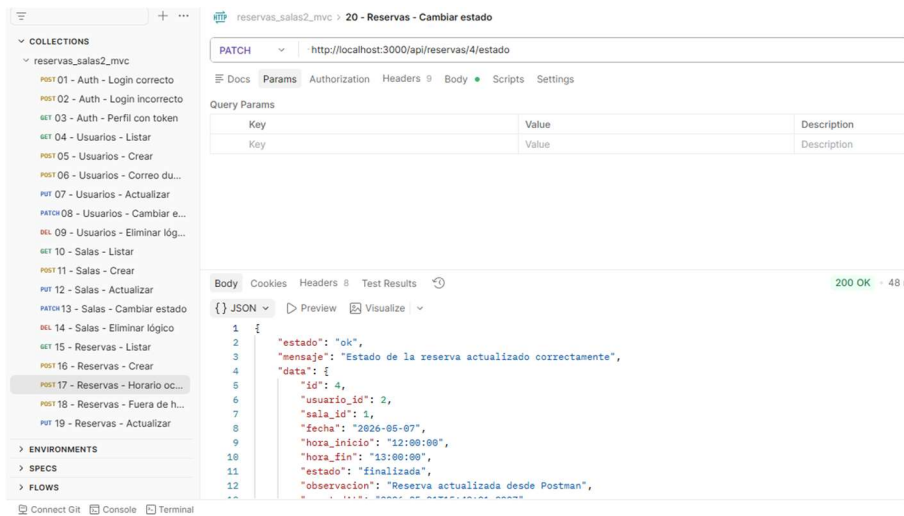
Método: PATCH

URL: `http://localhost:3000/api/reservas/4/estado`

Authorization: igual que las requests anteriores de reservas

Body → raw → JSON:

```
{
  "estado": "finalizada"
}
```



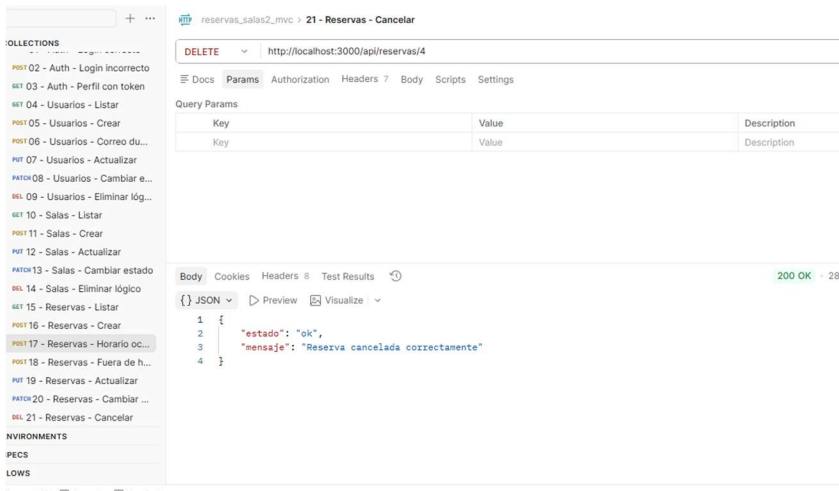
- **Prueba cancelar reserva**

Metodo : DELETE

URL: `http://localhost:3000/api/reservas/4`

Authorization: igual que las requests anteriores

Body: none



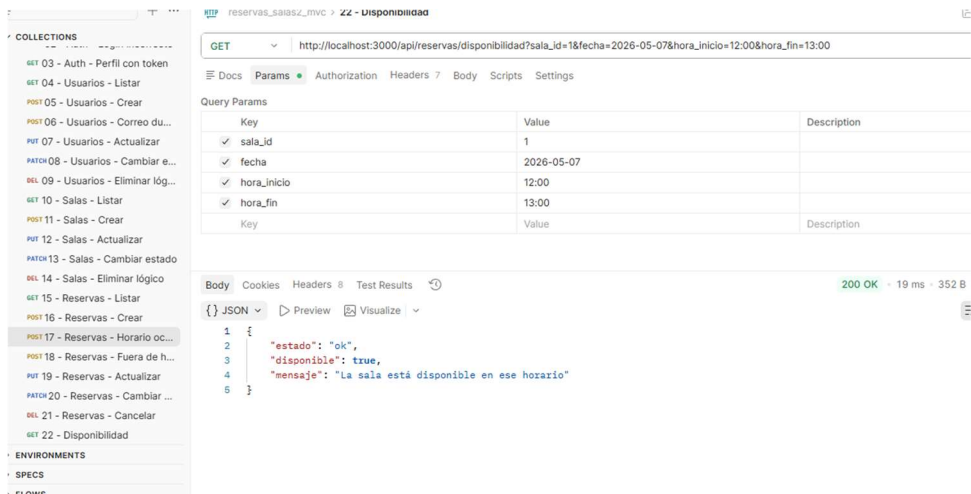
- **Disponibilidad**

Método: GET

URL: http://localhost:3000/api/reservas/disponibilidad?sala_id=1&fecha=2026-05-07&hora_inicio=12:00&hora_fin=13:00

Authorization:

Body: none



- Prueba de reserva ocupada

Metodo GET

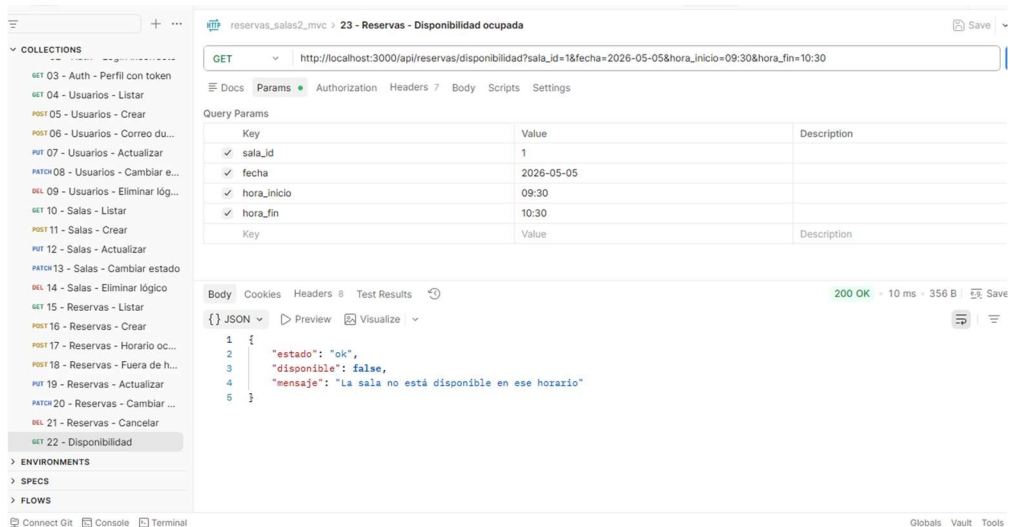
URL : http://localhost:3000/api/reservas/disponibilidad?sala_id=1&fecha=2026-05-05&hora_inicio=09:30&hora_fin=10:30

Authorization: igual que las requests anteriores

Body: none

Valor nuevo

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZiZCI6MSwiZW1haWwiOiJhZG1pbkByZXNlcnZhcj5jbCIsInJvbCI6ImFkbWluliwiaWF0IjoxNzc3NjU1MzkwLCJleHAiOiE3Nzc3NjU1MzkwOTB9.zhidUbAU1BmsfI7ajFO-iNPrpguz48B5MIQipxWpQt4



- **En .env se dejo :**

JWT_EXPIRES_IN=1h

Pasada 1 hora el token responderá algo como:

```
{  
  "estado": "error",  
  "mensaje": "Token inválido o expirado"  
}
```

8. Se crea README.md

Explica:

Cómo instalar el proyecto
Cómo crear la base de datos
Cómo correr migraciones
Cómo ejecutar seeders
Cómo levantar el servidor
Qué endpoints existen
Qué usuarios de prueba usar

9. Se crea .env.example

- Para subir proyecto a GitHub